

Documentação Técnica dos Scripts ETL (SGEF)

Membros

Bruno de Lima Marques

Caue Muniz Anastacio

Davi Pereira Araújo

**joão pedro nunes gomes da
silva**

Luisa de Moura Zimmer

SGEF(Sistema de Gestão e Educação Financeira)

1. Visão Geral do Processo

O processo de Engenharia de Dados foi dividido em três módulos principais (scripts Python conceituais) para garantir a modularidade e a rastreabilidade (ETL - Extração, Transformação e Carga).

Módulo (Script)	Função Principal	Saída
01_extrair_dados_form.py	Extração (E): Carrega o CSV do formulário e realiza a primeira inspeção.	df_bruto (DataFrame Bruto)
02_tratar_limpar_dados.py	Transformação (T): Renomeia colunas, padroniza strings e converte variáveis.	df_limpo (DataFrame Final)
03_estruturar_base_dados.py	Carga (L) e Modelagem: Desnormaliza o df_limpo em 4 tabelas modulares e extrai os dados do BCB.	5 Tabelas Finais (Prontas para Análise)

2. Detalhamento dos Scripts e Código

2.1. Módulo: Data_forms.pyinb (Extração) Primeira parte do codigo

Este script é responsável por localizar e carregar o arquivo CSV das respostas do Google Form, além de tratar erros comuns de caminho ou codificação.

```

import pandas as pd
from google.colab import drive
from IPython.display import display
from google.colab import files

drive.mount('/content/drive')

NOME_ARQUIVO_FORM = '/content/drive/MyDrive/respostas_formulario.csv'

def extrair_dados_form(nome_arquivo):
    """Carrega o arquivo CSV do Google Form."""
    try:
        df_bruto = pd.read_csv(nome_arquivo, sep=',', encoding='utf-8')

        print(f"\nSucesso! {len(df_bruto)} respostas encontradas no arquivo.")
        return df_bruto

    except FileNotFoundError:
        print(f"\nERRO DE EXTRAÇÃO: Arquivo '{nome_arquivo}' não encontrado.")
        print("Certifique-se de que o caminho acima está EXATO (incluindo letras maiúsculas/minúsculas da pasta).")
        return pd.DataFrame()
    except Exception as e:
        print(f"\nERRO DE LEITURA: Não foi possível ler o arquivo. Erro: {e}")
        return pd.DataFrame()

df_bruto = extrair_dados_form(NOME_ARQUIVO_FORM)

if not df_bruto.empty:
    print("\n✅ DataFrame Bruto carregado na variável 'df_bruto'.")
    print("Use os nomes das colunas abaixo para renomeá-las na Célula 2.")
    display(df_bruto.head())

```

Explicação: A função utiliza o `pd.read_csv` para carregar os dados. O uso do bloco `try...except` é uma boa prática de engenharia de dados, garantindo que o processo não falhe totalmente se o arquivo não estiver no local esperado.

2.2. Módulo: `Data_forms.pyinb` (Transformação) Segunda parte do código

Este é o módulo central, onde a limpeza e a padronização das respostas brutas são aplicadas, transformando o `df_bruto` em `df_limpo`.

```

df_limpo = df_bruto.copy()

df_limpo = df_bruto.rename(columns={
    'Faixa etária': 'faixa_etaria',
    'Participa de apostas': 'participa_apostas',
    'Reserva financeira': 'reserva_financeira',
    'Resgate imediato': 'resgate_imediato',
    'Poupança': 'poupanca',
    'Tesouro Nacional': 'tesouro',
    'Renda fixa': 'renda_fixa',
    'Renda variável': 'renda_variavel',
    'Reserva de valor': 'reserva_valor',
    'Nível de conhecimento': 'nivel_conhecimento',
    'Objetivo financeiro': 'objetivo_financeiro',
    'Decisão financeira': 'decisao_financeira',
    'Jornada financeira (opcional)': 'jornada_financeira',
    'Conselho financeiro (opcional)': 'conselho_financeiro',
    'E-mail (opcional)': 'email'
}, inplace=False)

for col in ['faixa_etaria', 'nivel_conhecimento', 'reserva_financeira', 'participa_apostas']:
    if col in df_limpo.columns:
        df_limpo[col] = df_limpo[col].astype(str).str.strip().str.title()

for col in ['jornada_financeira', 'conselho_financeiro']:
    if col in df_limpo.columns:
        df_limpo[col] = df_limpo[col].astype(str).str.strip()

if 'email' in df_limpo.columns:
    df_limpo['email'] = df_limpo['email'].astype(str).str.strip().str.lower()

CAMINHO_LOCAL = "respostas_formulario LIMPO.csv"
df_limpo.to_csv(CAMINHO_LOCAL, index=False, encoding="utf-8")
files.download(CAMINHO_LOCAL)

print("✓ Arquivo CSV gerado com sucesso:")
print(CAMINHO_LOCAL)

df_limpo.head()

```

Explicação: Esse parte do código renome todas as classes .cvs tiradas do df_bruto, transformando cada uma em nomes com letras minúsculas, após a renomeação ele executa um loop pegando secções específicas para pegar informações e transformá-los em string, como 'email', 'jornada_financeira' e 'conselho_financeiro'. Após todo esse processo ele pega as informações tiradas e transforma novamente em um arquivo .csv e manda para download na máquina de quem estiver executando o código. Todas as tabelas estão disponíveis na pasta de arquivo Data_csv.

2.2. Módulo: **BCB_Data.pyinb** (Transformação)

Ocorre o mesmo processo em um arquivo chamado BCB_Data.pyinb

```

import requests
import pandas as pd
from datetime import date
from IPython.display import display

# --- 1. DEFINIÇÃO DAS SÉRIES DO BCB ---
SERIES_BCB = [
    # DÍVIDA E RISCO
    {'codigo': 1441, 'nome': "Inadimplência PF Recursos Livres", 'area': 'Risco', 'unidade': '% ao Mês'},
    {'codigo': 25471, 'nome': "Juros Cheque Especial", 'area': 'Dívida', 'unidade': '% ao Ano'},
    {'codigo': 25473, 'nome': "Juros Cartão de Crédito", 'area': 'Dívida', 'unidade': '% ao Ano'},
    {'codigo': 25477, 'nome': "Juros Empréstimo Consignado", 'area': 'Dívida', 'unidade': '% ao Mês'},

    # MACROECONOMIA E Renda Fixa
    {'codigo': 433, 'nome': "IPCA Mensal (Inflação)", 'area': 'Inflação', 'unidade': '% ao Mês'},
    {'codigo': 4390, 'nome': "Taxa SELIC (Meta)", 'area': 'Renda Fixa', 'unidade': '% ao Ano'},
    {'codigo': 189, 'nome': "Poupança (Taxa Referencial)", 'area': 'Renda Fixa', 'unidade': '% ao Mês'},

    # REFERÊNCIAS DE MERCADO (O código 1 continua dando erro 406, mas será ignorado.)
    {'codigo': 1, 'nome': "Taxa de Câmbio Dólar (Venda - Livre)", 'area': 'Câmbio/Investimento', 'unidade': 'R/US$'},
    {'codigo': 7384, 'nome': "Índice de Confiança do Empresário", 'area': 'Sentimento', 'unidade': 'Pontos'},
]

def buscarSerieBcb(codigo_serie):
    """Busca os dados de uma série específica do BCB e retorna como lista de dicionários."""
    url = f'https://api.bcb.gov.br/dados/serie/bcdata.sgs.{codigo_serie}/dados?format=json'
    try:
        resposta = requests.get(url)
        resposta.raise_for_status()
        return resposta.json()
    except requests.exceptions.RequestException as e:
        print(f"ERRO: Não foi possível buscar o código {codigo_serie}. Erro: {e}")
        return []

def transformarDadosParaAnalise(dados_brutos, info_serie):
    """Limpa, estrutura e enriquece os dados brutos de uma série."""
    if not isinstance(dados_brutos, list) or not dados_brutos:
        return pd.DataFrame()

    df = pd.DataFrame(dados_brutos)
    df = df.rename(columns={'data': 'Data', 'valor': 'Valor'})

    df['Data'] = pd.to_datetime(df['Data'], format='%d/%m/%Y', errors='coerce')

    try:
        df['Valor'] = df['Valor'].astype(str).str.replace(',', '.', regex=True).astype(float)
    except AttributeError:
        df['Valor'] = df['Valor'].astype(float)

    df['Nome_Indicador'] = info_serie['nome']
    df['Area_Analise'] = info_serie['area']
    df['Unidade_Medida'] = info_serie['unidade']
    df['Fonte'] = 'Banco Central do Brasil (BCB)'

    return df[['Data', 'Nome_Indicador', 'Area_Analise', 'Valor', 'Unidade_Medida', 'Fonte']]

```

```

return df[['Data', 'Nome_Indicador', 'Area_Analise', 'Valor', 'Unidade_Medida', 'Fonte']]

def consolidar_base_final():
    """Executa o ETL completo e retorna a base de dados estruturada na memória."""
    lista_de_dfs = []
    print("Iniciando a Extração e Transformação de 9 séries do BCB...")

    for serie in SERIES_BCB:
        dados_json = buscar_serie_bcb(serie['codigo'])
        df_estruturado = transformar_dados_para_analise(dados_json, serie)

        if not df_estruturado.empty:
            lista_de_dfs.append(df_estruturado)
            print(f"-> SUCESSO: Série {serie['codigo']} - {serie['nome']} estruturada.")
        else:
            print(f"-> AVISO: Série {serie['codigo']} - {serie['nome']} não pôde ser incluída.")

    if not lista_de_dfs:
        print("\nFALHA: Nenhuma base de dados pôde ser consolidada.")
        return pd.DataFrame()

    base_de_dados_final = pd.concat(lista_de_dfs, ignore_index=True)

    nome_arquivo = f'base_bcb_estruturada_{date.today()}.csv'
    base_de_dados_final.to_csv(nome_arquivo, index=False, sep=';', encoding='utf-8')

    print("\n--- BASE DE DADOS ESTRUTURADA CONCLUÍDA ---")
    print(f"Arquivo salvo como: {nome_arquivo}")
    print(f"Total de registros na base consolidada: {base_de_dados_final.shape[0]}")

    return base_de_dados_final

df_principal = consolidar_base_final()

```

Explicação: Esse código realiza as duas fases de extração e transformação, repetindo o mesmo processo de pegar dados que estão armazenados na API do Banco Central do Brasil que estão em arquivo .csv nomeados em formas de números. Em seguida faz uma renomeação para facilitar o tratamento de dados e em seguida gera um arquivo um formato .csv em quem estiver executando-o.

2.3. Módulo: **03_estruturar_base_dados.py** (Modelagem e Carga)

Este script finaliza a modelagem de dados, desmembrando o **df_limpo** em tabelas modulares (aplicando o Princípio da Responsabilidade Única) e conectando os metadados do BCB e do forms.

```
1  import os
2  import sys
3  import pandas as pd
4
5  script_dir = os.path.dirname(os.path.abspath(__file__))
6  project_root = os.path.dirname(script_dir)
7  PASTA_DATABASE = os.path.join(project_root, 'database')
8  os.makedirs(PASTA_DATABASE, exist_ok=True)
9  print(f"Diretório de saída: {PASTA_DATABASE}")
10
11 caminho_csv = os.path.join(PASTA_DATABASE, 'respostas_formulario.csv')
12 print(f"Tentando carregar: {caminho_csv}")
13
14
15 df = None
16 for sep, enc in [(';','utf-8-sig'), (',','utf-8-sig'), (';','latin-1')]:
17     try:
18         df = pd.read_csv(caminho_csv, sep=sep, encoding=enc)
19         print(f"Lido com sep='{sep}' enc='{enc}'")
20         break
21     except FileNotFoundError:
22         print("ERRO: arquivo não encontrado:", caminho_csv)
23         sys.exit(1)
24     except Exception:
25         df = None
26
27 if df is None:
28     print("ERRO: não foi possível ler o CSV. Verifique separador/encoding.")
29     sys.exit(1)
30
31 print("\n--- primeiras linhas ---")
32 print(df.head().to_string(index=False))
33 print("\n--- colunas originais ---")
```

```

print(list(df.columns))

df.columns = (
    df.columns
    .astype(str)
    .str.replace('\uffeff', '', regex=False)
    .str.strip()
    .str.lower()
    .str.normalize('NFKD')
    .str.encode('ascii', errors='ignore').str.decode('ascii')
    .str.replace(r'\W+', '_', regex=True)
    .str.strip('_')
)

print("\n-- colunas normalizadas --")
print(list(df.columns))

colunas_perfil = ['faixa_etaria', 'nivel_conhecimento', 'objetivo_financeiro', 'email']
colunas_comportamento = ['participa_apostas', 'reserva_financeira', 'resgate_imediato', 'decisao_financeira']
colunas_investimentos = ['poupanca', 'tesouro', 'renda_fixa', 'renda_variavel', 'reserva_valor']
colunas_textos = ['jornada_financeira', 'conselho_financeiro']

def selecionar_colunas(df, lista):
    existentes = [c for c in lista if c in df.columns]
    faltantes = [c for c in lista if c not in df.columns]
    if faltantes:
        print(f"Aviso: colunas ausentes e serão ignoradas: {faltantes}")
    if not existentes:
        print(f"Aviso: nenhuma das colunas esperadas {lista} encontrada.")
    return df.iloc[0:0].copy()

```

```

    return df.iloc[0:0].copy()
    return df[existentes].copy()

tabela_perfil_pessoal = selecionar_colunas(df, colunas_perfil)
tabela_comportamento = selecionar_colunas(df, colunas_comportamento)
tabela_investimentos = selecionar_colunas(df, colunas_investimentos)
tabela_textos_livres = selecionar_colunas(df, colunas_textos)

▼ tabelas_para_salvar = {
    'tabela_perfil_pessoal': tabela_perfil_pessoal,
    'tabela_comportamento': tabela_comportamento,
    'tabela_investimentos': tabela_investimentos,
    'tabela_textos_livres': tabela_textos_livres
}

for nome, df_salvar in tabelas_para_salvar.items():
    caminho_arquivo = os.path.join(PASTA_DATABASE, f'{nome}.csv')
    if df_salvar.empty:
        print(f"Pulado: {nome}.csv (vazio) - não será salvo.")

        continue

    df_salvar.to_csv(caminho_arquivo, index=False, sep=';', encoding='utf-8')
    print(f"📁 {nome}.csv salvo ({len(df_salvar)} linhas) em {caminho_arquivo}")

print("✅ Finalizado.")

```

Explicação: O código final utiliza a biblioteca pandas para fazer a modelagem dos dados retirados dos 2 documentos de dados fornecidos e são o produto final do ETL, sendo a

fonte direta para o Relatório de Insights e quaisquer visualizações de BI. Esse é um arquivo .py, porém minha recomendação pessoal é entrar nos arquivos **BCB_Data.pyinb** e **Data_forms.pyinb** , que possui todas as fases do processo de ETF, inclusive a modelagem de dados e a geração de gráficos.